Classification Reporting & Analytics — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Classification Reporting & Analytics (Microsoft Purview) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Classification Reporting & Analytics (Microsoft Purview) 5

What this solution is . . . 5

Content Explorer vs Activity Explorer in one paragraph each . . . 5

How the reporting signal is produced . . . 6

What this delivers . . . 6

What is authored exactly vs deferred / flagged . . . 6

The standard deployment model . . . 7

2. Requirements Guide — Classification Reporting & Analytics (Microsoft Purview) 7

2.1 Licensing that unlocks the explorers (category GR1) . . . 7

2.2 Least-privilege reporting roles — the two Content Explorer viewer role groups (category GR2) . 8

2.3 A named reporting-analyst group and role separation . . . 8

2.4 Audit ingestion enabled (Activity Explorer's data source) . . . 8

2.5 Existing classification signal to report on (in-scope data — category GR4) . . . 8

2.6 A pilot reporting scope and change control (category GR6) . . . 9

2.7 PowerShell toolchain — S&C; PowerShell + Exchange Online + Graph (category GR3) . . . 9

Global Requirements — Microsoft Purview Solutions 9

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 10

2. Purview roles (least-privilege) and portal access . . . 10

3. Security & Compliance PowerShell + Graph toolchain . . . 10

4. Data sources & scope in scope . . . 10

5. Compliance, records, privacy & legal stakeholders . . . 11

6. A pilot scope and change control . . . 11

3. Implementation Guide — Classification Reporting & Analytics (Microsoft Purview) 11

Phase 1 — Assess (read-only, no changes) . . . 12

Phase 2 — Design . . . 13

Phase 3 — Configure (governed; pilot first) . . . 13

Phase 4 — Pilot & Validate . . . 14

Phase 5 — Expand & Operate . . . 15

5. Testing & Validation Guide — Classification Reporting & Analytics (Microsoft Purview) 15

Test 1 — Licensing / surface availability . . . 16

Test 2 — Audit ingestion enabled . . . 16
Test 3 — Least-privilege role membership is correct . . . 16
Test 4 — Content Explorer returns coverage data . . . 16
Test 5 — Activity Explorer / audit returns activity . . . 17
Test 6 — Dashboard renders both views . . . 17
Test 7 — Revocation works (rollback proof) . . . 17
Test 8 — Scheduled refresh + health check . . . 17
Exit criteria (all must hold, with the two expected-states allowed) . . . 17
6. Troubleshooting Guide — Classification Reporting & Analytics (Microsoft Purview) 18
Content Explorer or Activity Explorer is missing, or shows only aggregate cards . . . 18
The explorer isn't where this guide says (menu path differs) . . . 18
`CLSREPORT-Configure.ps1` made no change . . . 18
Audit ingestion shows False (Activity Explorer / activity export empty) . . . 19
`Search-UnifiedAuditLog` returns nothing or errors . . . 19
`CLSREPORT-Report.ps1` coverage-by-tag.csv is empty or all VERIFY . . . 19
`CLSREPORT-Validate.ps1` reports FAIL on role membership . . . 19
An analyst can see item lists but not open content (or vice versa) . . . 20
Revocation didn't take effect . . . 20
`CLSREPORT-ScopeHealth.ps1` shows a workload "not reporting in" . . . 20
`Export-ContentExplorerData` / `Export-ActivityExplorerData` "not recognized" . . . 20
The dashboard tiles broke after a script run . . . 20
The solution "changed" something . . . 20
Escalation boundary . . . 21
7. Best Practices — Classification Reporting & Analytics (Microsoft Purview) 21
8. Deliverables Summary — Classification Reporting & Analytics (Microsoft Purview) 22
Documentation . . . 22
PowerShell scripts . . . 23
What the buyer receives . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Classification Reporting & Analytics (Microsoft Purview)

Leadership keeps asking two questions that most organizations cannot answer with evidence: **"What is classified, where is it, and how much of our estate is covered?"** and **"What are people actually doing with our sensitive and labeled content?"** Microsoft Purview already holds the answers in **Content Explorer** and **Activity Explorer** — but only if the reporting surfaces are enabled, scoped to the right people under least privilege, and read into repeatable dashboards. This solution is a repeatable, self-guided implementation that stands that reporting up end to end, using only the Microsoft Purview portal, Security & Compliance PowerShell, and Exchange Online PowerShell — no third-party tooling.

It is **read-and-report by design.** It does not decide your classification scheme, create sensitivity or retention labels, or author DLP policy — those are separate Purview solutions, and where they carry regulatory or records weight their determinations stay with compliance, records, privacy, and legal. Here you make the *existing* classification signal **visible, measurable, and dashboard-ready**. The only governed changes it makes — enabling unified audit log ingestion and granting the scoped viewer roles — are **previewed with** `-WhatIf` **and applied only with** `-Commit` under change control, and piloted on a small analyst subset before the full audience.

What you get

- A plain-language **Solution Overview** of what the reporting delivers — the coverage view (Content

Explorer) and the behavior/trend view (Activity Explorer) — with the availability/licensing caveat for these evolving, plan-gated surfaces, and the boundary between reporting and enforcement.

- A **Requirements Guide** with verify / implement-if-missing for every prerequisite — compliance

licensing, the two least-privilege Content Explorer viewer role groups, a named analyst group with role separation, audit ingestion, the classification signal to report on, and a pilot scope + change control.

- An **Implementation Guide** of precise, ordered portal and PowerShell steps: baseline readiness, confirm

the signal, design the analyst group and dashboard, enable audit + the scoped viewer roles (pilot first), open the explorers, produce the pilot reports, validate against pass/fail gates, expand to the full audience, and schedule the exports and health checks.

- **Five PowerShell scripts** — Discovery, Configure (audit + viewer roles, `-WhatIf`/`-Commit`),

Validate, Report (coverage + activity CSV exports), and ScopeHealth — read-only or `-WhatIf` by default, with no credentials in code.

- **Testing and Troubleshooting** guides with explicit pass/fail gates, where an empty/baseline estate and

audit-ingestion lag are treated as expected states, not failures.

What this delivers

A **coverage report** of what carries each sensitivity label, retention label, sensitive information type, and trainable classifier, and where; an **activity report** of label lifecycle, DLP matches, and egress over a rolling

window; **dashboards** (Excel or Power BI at the buyer's choice) fed by repeatable CSV exports; and the **least-privilege, audited role model** underneath — every step read-only or previewed before commit, changing no content.

The deployment model

Assess → Design → Configure → Pilot (simulate/limited scope) → Validate → Expand — with explicit pass/fail gates and a documented rollback (revoke the viewer role membership).

Outcome

Classification made **visible, measurable, and defensible** in Microsoft Purview — coverage and activity reported, dashboards standing, and access least-privilege and audited — so leadership can see what is classified and where, and prove the classification program is working. Feature availability is flagged to verify in your tenant; regulatory, records, and legal determinations arising from the reports remain with your governance functions. This solution delivers the technical method and the evidence, not the ruling.

1. Solution Overview — Classification Reporting & Analytics (Microsoft Purview)

What this solution is

This solution stands up **classification reporting and analytics** in **Microsoft Purview** using the two native explorers: **Content Explorer** (where classified content lives right now) and **Activity Explorer** (what has happened to labeled/sensitive content over time). The outcome is a defensible, repeatable answer to the questions leadership keeps asking: *"What is classified, where is it, and how much of our estate is covered?"* and *"What are people doing with our sensitive and labeled content?"*

It covers the whole path: confirming the **licensing and least-privilege roles** that gate the explorers, **enabling** the reporting surfaces (audit ingestion + the Content Explorer viewer role groups scoped to a named reporting group), **piloting** the reporting on a limited scope, **validating** that both explorers return data, and **operating** it with repeatable PowerShell exports and dashboards. It is written for an administrator with no prior Purview-reporting experience.

It is **read-and-report by design.** This solution does **not** decide your classification scheme, create sensitivity/retention labels, or author DLP policy — those are separate Purview solutions and, where they carry regulatory or records weight, their **determinations are deferred to compliance, records, privacy, and legal**. Here you make the *existing* classification signal **visible, measurable, and dashboard-ready**.

Content Explorer vs Activity Explorer in one paragraph each

- **Content Explorer** is a **current-state snapshot** of your estate. It shows, per location (Exchange mailboxes, SharePoint sites, OneDrive accounts, and — where supported — Teams), the items where Purview has detected a **sensitive information type (SIT)**, a **sensitivity label**, a **retention label**, or a **trainable classifier**. You drill from an aggregate count down to the individual file or message. It answers **"what is classified and where, right now."**

• **Activity Explorer** is a **time-series of activity** (a rolling window, typically up to ~30 days —

verify your tenant). It shows events on labeled/sensitive content: **label applied, changed, downgraded, or removed**; **DLP rule matches**; **files copied to removable media / network share / cloud**; and more, filterable by location, user, label, and activity type. It answers **"what is happening to our classified content over time."**

Together they give the **coverage** view (Content Explorer) and the **behavior/trend** view (Activity Explorer) that a classification program needs to prove it is working.

How the reporting signal is produced

• **Content Explorer** is populated by Purview's classification engine as it scans and labels content —

you don't feed it; you **grant sight of it** (the viewer role groups) and **read** it. In PowerShell the same data is exportable with `Export-ContentExplorerData` (Security & Compliance PowerShell), which returns items by tag type (SIT / Label / RetentionLabel / TrainableClassifier) — used here for repeatable coverage exports. *(Availability is licensing-gated — verify your tenant.)*

• **Activity Explorer** is populated from the **unified audit pipeline**. The portal surface is Activity

Explorer; the same activity is queryable in PowerShell with `Search-UnifiedAuditLog` filtered to the sensitivity-label, DLP, and endpoint record types — used here for repeatable activity exports and trend dashboards. **Audit ingestion must be enabled** for either surface to be complete.

What this delivers

• **Coverage reporting** — counts and locations of content carrying each **sensitivity label**,

retention label, **sensitive information type**, and **trainable classifier**, exportable to CSV and chartable — the "how much of the estate is classified, and where" answer.

• **Activity reporting** — label lifecycle (apply/change/downgrade/remove), DLP matches, and egress

activities over a rolling window — the "what are people doing with it" answer.

• **Dashboards** — the exports feed a repeatable dashboard (Excel/Power BI at the buyer's choice) so the

numbers are shown, not just queried. *(Power BI/Fabric modeling is referenced at the pattern level; its licensing and build are the buyer's — this solution delivers the data and the shape.)*

• **Operational hygiene** — least-privilege viewer roles scoped to a named analyst group, audit-lag and

role-drift health checks, and a pilot-first rollout.

What is authored exactly vs deferred / flagged

• **Exact navigation and cmdlets** are given for the **Microsoft Purview portal**

(`purview.microsoft.com`), the **classic compliance portal** (`compliance.microsoft.com`) where a surface still lives there, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and where useful **Microsoft Graph**.

• **Licensing is plan-gated and flagged.** Content Explorer content viewing, Activity Explorer, and

trainable classifiers generally require **Microsoft 365 E5 / E5 Compliance** — availability is **"verify your licensing/tenant"**, never assumed.

• **The portal is evolving.** Content Explorer and Activity Explorer are moving between the classic compliance portal and the new Purview portal, and card/menu names shift. Where a path is version-dependent this solution gives both known locations and flags **"verify against your current portal."**

• **Determinations are deferred.** What the classification scheme *should* be, what retention a record *must* have, whether an activity is a *policy violation* — those belong to compliance, records, privacy, and legal. This solution reports the signal and provides the evidence; it does not rule.

The standard deployment model

> ***Assess*** *licensing + roles + what is already classified →* ***Design*** *the reporting scope, the analyst role group, and the dashboard shape →* ***Configure*** *audit ingestion + the Content Explorer viewer role groups (scoped) →* ***Pilot*** *the reporting on a limited scope →* ***Validate*** *both explorers return data against pass/fail gates →* ***Expand*** *to the full analyst audience and scheduled exports. Pattern:* ***Assess*** *→* ***Design*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand****, with explicit pass/fail gates and a documented rollback. Role and audit changes are handled under change control and piloted before broad grant.*

2. Requirements Guide — Classification Reporting & Analytics (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why required · How to verify · How to implement if missing.** The category-wide Purview prerequisites (compliance licensing, least-privilege Purview roles, Security & Compliance PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope + change control) are in `global-requirements-purview.md` — confirm those **six first**; this guide adds the reporting-specific detail and does not duplicate them.

> ***Object names used throughout (use verbatim):*** *analyst security group* `Purview Classification Reporting - Analysts` *(Entra group holding the people who read the explorers); built-in Purview role groups* `Content Explorer List Viewer` *and* `Content Explorer Content Viewer`*; pilot reporting scope label* `Classification Reporting - Pilot`*; export output folder* `.\CLSREPORT-out`*. No labels, SITs, or policies are created by this solution — it reports on whatever already exists.*

2.1 Licensing that unlocks the explorers (category GR1)

Description: A Microsoft 365 tenant licensed for **Data Classification** analytics. **Content Explorer** aggregate cards are broadly available, but **viewing item content**, **Activity Explorer**, and **trainable-classifier** signal generally require **Microsoft 365 E5** or the **E5 Compliance** add-on. **Why required:** The explorers are plan-gated; without the right plan the surface is missing or shows only partial (aggregate-only) data. **How to verify:** M365 admin center → confirm the E5 / E5 Compliance subscription; in the Purview portal confirm **Content explorer** and **Activity explorer** are present and populated. Run `CLSREPORT-Discovery.ps1` — it reports the licensing-dependent signals it can see. **How to**

implement if missing: Assign/purchase the required licensing before relying on item-level or activity reporting. Flag **"verify your licensing/tenant"** in any report footnote where a plan gate applies.

2.2 Least-privilege reporting roles — the two Content Explorer viewer role groups (category GR2)

Description: Reading Content Explorer requires membership in `Content Explorer List Viewer` (see the **list** of items and their locations) and/or `Content Explorer Content Viewer` (open and **read the item content**). Activity Explorer is visible to compliance/data-admin roles (e.g. **Compliance Administrator**, **Compliance Data Administrator**, Information Protection roles). **Why required:** Purview separates *seeing that content exists* from *reading it*; least-privilege reporting means analysts get the minimum they need, and **Content Viewer** (which exposes actual sensitive data) is granted deliberately and sparingly. **How to verify:** Purview portal → **Roles & scopes** → **Permissions** (or `Get-RoleGroupMember` in S&C PowerShell) shows membership of the two role groups; `CLSREPORT-Discovery.ps1` reports current members. **How to implement if missing:** Add the analyst group (or named individuals) to the least-privilege role group under change control — `Content Explorer List Viewer` for coverage reporting, and `Content Explorer Content Viewer` only for those who must inspect the sensitive items. Handled by `CLSREPORT-Configure.ps1` (`-WhatIf` first).

2.3 A named reporting-analyst group and role separation

Description: An Entra security group `Purview Classification Reporting - Analysts` that holds the people authorized to read classification reports, kept **separate** from the people who *author* labels/DLP and separate again from records/legal. **Why required:** Content Explorer content viewing exposes real sensitive data — access must be least-privilege, auditable, and role-separated; a named group makes grants and revocations clean and reviewable. **How to verify:** The group exists with the intended members; role-group membership points at the group (or its members), not at "Everyone" or a broad admin group. **How to implement if missing:** Create the group, populate it from the stakeholder list, and target the role-group membership at it.

2.4 Audit ingestion enabled (Activity Explorer's data source)

Description: Unified audit log ingestion must be **on** for Activity Explorer and for the `Search-UnifiedAuditLog`-based activity exports to be complete. **Why required:** Activity Explorer is populated from the audit pipeline; with ingestion off, activity reporting is empty or partial. **How to verify:** `Get-AdminAuditLogConfig | Select UnifiedAuditLogIngestionEnabled` (Exchange Online PowerShell) returns **True**; a `Search-UnifiedAuditLog` for a recent window returns rows. `CLSREPORT-Discovery.ps1` reports the ingestion state. **How to implement if missing:** Enable it (`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`) under change control — surfaced by `CLSREPORT-Configure.ps1` (`-Commit` gated). Allow time for the pipeline to backfill before judging coverage.

2.5 Existing classification signal to report on (in-scope data — category GR4)

Description: There must be **published sensitivity labels**, **retention labels**, **sensitive information types**, and/or **trainable classifiers** in the tenant, applied across the in-scope workloads (Exchange,

SharePoint, OneDrive, Teams), for the explorers to have anything to show. **Why required:** Reporting is only as rich as the classification underneath it; a tenant with no published labels or SIT detections will show near-empty explorers. **How to verify:** `Get-Label`, `Get-ComplianceTag`, `Get-DlpSensitiveInformationType` return the expected inventory; Content Explorer shows non-zero counts. `CLSREPORT-ScopeHealth.ps1` inventories what is available to report on. **How to implement if missing:** Classification design/roll-out is a **separate** solution — this one reports on the result. If the estate is largely unclassified, note that the coverage numbers are a **baseline**, not a failure, and reference the labeling/SIT solutions.

2.6 A pilot reporting scope and change control (category GR6)

Description: A limited **pilot scope** — a subset of sites/mailboxes or a defined reporting boundary tagged `Classification Reporting - Pilot` — to prove the reporting and dashboards on before granting the full analyst audience, plus a **change-control** path for the role/audit changes. **Why required:** Reporting itself is low-risk, but **granting Content Explorer Content Viewer** (which exposes sensitive data) and **changing audit config** are governed actions that must be piloted and approved. **How to verify:** A named pilot scope and approver exist; the initial role grant targets a small pilot analyst set before the full group. **How to implement if missing:** Define the pilot scope and change window before running `CLSREPORT-Configure.ps1 -Commit`.

2.7 PowerShell toolchain — S&C PowerShell + Exchange Online + Graph (category GR3)

Description: A workstation with the **ExchangeOnlineManagement** module (provides both `Connect-IPPSSession` for Security & Compliance PowerShell and `Connect-ExchangeOnline` / `Search-UnifiedAuditLog` / `Get-AdminAuditLogConfig`) and, optionally, **Microsoft Graph PowerShell**, with a modern-auth / MFA-capable admin. **Why required:** The five provided scripts and the exports run through these; `Export-ContentExplorerData` and `Search-UnifiedAuditLog` live in this toolchain. **How to verify:** `Connect-IPPSSession` and `Connect-ExchangeOnline` succeed; `Get-Command Export-ContentExplorerData` resolves (its availability is tenant/version-dependent — **verify**). **How to implement if missing:** `Install-Module ExchangeOnlineManagement` (and `Microsoft.Graph`), confirm modern-auth sign-in, then run the scripts.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention,*

> ***and legal determinations*** *(retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are* ***deferred to the organization's compliance, records, privacy, and legal functions*** *— the solutions provide the technical configuration method and evidence, not the determination. Automation is* ***PowerShell*** *(Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Classification Reporting & Analytics (Microsoft Purview)

Follow these steps **in order**. Portal navigation is given for both the **new Microsoft Purview portal** (`https://purview.microsoft.com`) and the **classic compliance portal** (`https://compliance.microsoft.com`) because Content Explorer / Activity Explorer are **actively moving between them** — where a path is version-dependent, both are named and you **verify against your current portal**. PowerShell is **Security & Compliance PowerShell** (`Connect-IPPSSession`) plus **Exchange Online PowerShell** (`Search-UnifiedAuditLog`, `Get/Set-AdminAuditLogConfig`).

Consoles used:

- **Microsoft Purview portal** — `https://purview.microsoft.com`
- **Classic compliance portal** — `https://compliance.microsoft.com` (fallback for a surface not yet moved)
- **Security & Compliance PowerShell** — `Connect-IPPSSession`
- **Exchange Online PowerShell** — `Connect-ExchangeOnline` (audit config + unified audit log)

> ***Model:*** *Confirm licensing/roles; enable audit + the scoped viewer roles;* ***pilot*** *the reporting on* `Classification Reporting - Pilot`*; validate both explorers return data; then* ***expand*** *to the full analyst group and scheduled exports.* ***Assess*** *→* ***Design*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***
>
> ***Artifact names (use verbatim):*** *analyst group* `Purview Classification Reporting - Analysts`*; role groups* `Content Explorer List Viewer` *and* `Content Explorer Content Viewer`*; pilot scope* `Classification Reporting - Pilot`*; export folder* `.\CLSREPORT-out`*.*
>
> ***Risk + rollback (read first):*** *This solution is* ***read-and-report*** *— it creates* ***no*** *labels, SITs, or policies, so it cannot mislabel or block content. The two governed actions are (a)* ***granting*** `Content Explorer Content Viewer`*, which exposes real sensitive data to the grantee, and (b)* ***enabling unified audit log ingestion****. Both are piloted and change-controlled.* ***Rollback:*** *remove members from the role group (*`Remove-RoleGroupMember`*) to revoke reporting access — revocation takes effect on next sign-in / token refresh; audit ingestion, once on, is normally left on (disabling it loses activity history — treat as a separate decision). No content is ever modified.*

Phase 1 — Assess (read-only, no changes)

Step 1 — Baseline the reporting readiness

1. Connect: `Connect-IPPSSession` and `Connect-ExchangeOnline`.
2. Run `CLSREPORT-Discovery.ps1`. It reports, read-only:

- licensing-dependent signals it can see (whether Content/Activity Explorer data is returnable);
- current membership of `Content Explorer List Viewer` and `Content Explorer Content Viewer`;
- `UnifiedAuditLogIngestionEnabled` state;
- an inventory count of **sensitivity labels**, **retention labels**, and **sensitive information types** available to report on.

1. **Pass/fail gate:** you have a written baseline of licensing, roles, audit state, and available

classification signal. If audit ingestion is **False** or no labels/SITs exist, resolve those (Requirements 2.4 / 2.5) before expecting meaningful reports.

Step 2 — Confirm the classification signal to report on

1. Run `CLSREPORT-ScopeHealth.ps1`. It lists the **published** sensitivity labels (`Get-Label`),

retention labels (`Get-ComplianceTag`), and SITs (`Get-DlpSensitiveInformationType`) — the tag set Content Explorer can break down by — and flags health signals (audit lag, empty inventory).

1. **Pass/fail gate:** a non-empty inventory of tags to report on, or an explicit note that the estate is

a **baseline** (largely unclassified) — a valid finding, not a failure.

Phase 2 — Design

Step 3 — Design the analyst group and role split

1. In Entra, confirm/create the security group `Purview Classification Reporting - Analysts` and

populate it from the stakeholder list (Requirement 2.3).

1. Decide the **role split**: everyone who needs coverage reporting → `Content Explorer List Viewer`;

only those who must **open sensitive items** → `Content Explorer Content Viewer` (kept minimal). Record who gets which and why (least-privilege justification).

1. Decide the **pilot analyst subset** (a few named people) for the first grant.

Step 4 — Design the dashboard shape

1. Decide the two dashboard views: **Coverage** (from `Export-ContentExplorerData` — items per label /

retention label / SIT / classifier, by location) and **Activity** (from `Search-UnifiedAuditLog` — label lifecycle + DLP matches + egress over a rolling window).

1. Choose the surface: **Excel** (bundled workbook from the CSV exports) for a quick start, or **Power

BI / Fabric** for a live model. *(Power BI licensing/build is the buyer's; this solution delivers the CSVs and the column shape the model binds to — pattern-level, not a determination.)*

Phase 3 — Configure (governed; pilot first)

Step 5 — Enable unified audit log ingestion (if not already on)

1. Preview first: `CLSREPORT-Configure.ps1 -EnableAudit -WhatIf` — shows the intended change.

2. When approved under change control: `CLSREPORT-Configure.ps1 -EnableAudit -Commit`. This runs

`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`.

1. **Pass/fail gate:** `Get-AdminAuditLogConfig` shows **True**. Allow the pipeline time to populate

before judging activity coverage.

Step 6 — Grant the scoped Content Explorer viewer roles (pilot subset)

1. Preview: `CLSREPORT-Configure.ps1 -GrantListViewer -Members <pilot analysts> -WhatIf` and, for

those who need content, `-GrantContentViewer -Members <minimal set> -WhatIf`.

1. When approved: re-run with `-Commit`. This calls `Add-RoleGroupMember -Identity "Content Explorer

List Viewer" (and "Content Explorer Content Viewer"`) for the named members.

1. **Pass/fail gate:** `Get-RoleGroupMember` shows the pilot analysts in the intended role group(s), and

only the minimal set in **Content Viewer**.

Step 7 — Open the explorers as a pilot analyst

1. As a pilot analyst, open **Content Explorer**:

• New portal: **Purview portal** → **Information Protection** → **Content explorer** (or **Data loss

prevention → Explorers**);

• Classic: **compliance.microsoft.com** → **Data classification** → **Content explorer**.

Confirm you can see counts by **label / retention label / SIT / classifier** and drill to a location.

1. Open **Activity Explorer**:

• New portal: **Purview portal** → **Data loss prevention** → **Activity explorer** (or under **Information

Protection**); Classic:** compliance.microsoft.com → Data classification → Activity explorer**. Confirm recent activity rows appear and filter by** activity type **and** label**.

1. **Verify against your current portal** — if a menu differs, the surface has moved; the data is the same.

Phase 4 — Pilot & Validate

Step 8 — Produce the pilot reports

1. Run `CLSREPORT-Report.ps1 -Days 30 -OutputFolder .\CLSREPORT-out`. It writes:

• `coverage-by-tag.csv` — items per sensitivity label / retention label / SIT (from

`Export-ContentExplorerData`, where licensed);

• `activity-summary.csv` — label lifecycle + DLP + egress activity counts over the window (from

`Search-UnifiedAuditLog`).

1. Load the CSVs into the chosen dashboard (Excel workbook or Power BI). Confirm the two views render.

Step 9 — Validate against pass/fail gates

1. Run `CLSREPORT-Validate.ps1`. It checks and reports PASS/FAIL for:

• audit ingestion **enabled**;

• the pilot analysts present in the intended role group(s), Content Viewer minimal;

• Content Explorer export returns ≥ **1** item for at least one tag (or an explicit "baseline / empty estate" note);

• `Search-UnifiedAuditLog` returns ≥ **1** classification/DLP activity row in the window (or a "no activity yet — allow ingestion time" note).

1. **Gate:** all applicable checks **PASS** (empty-estate and ingestion-lag are explained, not failed) before expanding.

Phase 5 — Expand & Operate

Step 10 — Expand to the full analyst audience

1. Re-run `CLSREPORT-Configure.ps1 -GrantListViewer -Members <full analyst group> -Commit` (and Content Viewer only for the minimal approved set) to move from the pilot subset to the full `Purview Classification Reporting - Analysts` audience.

1. **Pass/fail gate:** `Get-RoleGroupMember` matches the approved audience; no unintended broad grants.

Step 11 — Schedule the reports and health checks

1. Schedule `CLSREPORT-Report.ps1` (e.g. weekly) to refresh `.\CLSREPORT-out` for the dashboard.

2. Schedule `CLSREPORT-ScopeHealth.ps1` (e.g. weekly) to catch **role-group drift** (unexpected new members of Content Viewer), **audit lag**, and **new/removed labels** that change the report shape.

1. Review with compliance/records/privacy on a cadence. **Determinations** arising from the reports (e.g. "this content must be relabeled/retained/investigated") are **routed to the owning governance function** — this solution provides the evidence, not the ruling.

5. Testing & Validation Guide — Classification Reporting & Analytics (Microsoft Purview)

Run these tests after implementation and on a cadence thereafter. Each test has a **procedure**, a **pass** signal, and a **fail** signal. Two conditions are **expected states, not failures** and are called out explicitly: an **empty/baseline estate** (little is classified yet) and **ingestion lag** (audit just enabled). The automated gate is `CLSREPORT-Validate.ps1`; the manual tests confirm the portal experience.

> ***Golden rule:*** *this solution reports; it does not classify. A low coverage number is a* ***finding to route to governance****, not a test failure. Tests validate that the *reporting* is correct and complete, not that the *estate* is fully classified.*

Test 1 — Licensing / surface availability

- **Procedure:** Open Content Explorer and Activity Explorer in the portal (paths in Implementation Step 7). Run `CLSREPORT-Discovery.ps1`.
- **Pass:** Both explorers load; item-level content and activity are visible (E5/E5 Compliance present).
- **Fail:** A surface is missing or only aggregate cards show → **verify your licensing/tenant** (the plan gate). Note the gate in the report footnote; do not treat aggregate-only as a bug.

Test 2 — Audit ingestion enabled

- **Procedure:** `Get-AdminAuditLogConfig | Select UnifiedAuditLogIngestionEnabled`, or read the Discovery output.
- **Pass:** `True`.
- **Fail:** `False` → enable via `CLSREPORT-Configure.ps1 -EnableAudit -Commit` (change-controlled), then re-test after the pipeline populates.

Test 3 — Least-privilege role membership is correct

- **Procedure:** `CLSREPORT-Validate.ps1` (or `Get-RoleGroupMember "Content Explorer List Viewer"` and `... "Content Explorer Content Viewer"`).
- **Pass:** The intended analysts are in **List Viewer**; only the **minimal approved set** is in **Content Viewer**; no broad/unexpected members.
- **Fail:** Unexpected members (especially in Content Viewer, which exposes sensitive data) → remove with `Remove-RoleGroupMember` and investigate how they were added (role-group drift).

Test 4 — Content Explorer returns coverage data

- **Procedure:** Run `CLSREPORT-Report.ps1 -Days 30`; open `coverage-by-tag.csv`. In the portal, drill a label/SIT card to a location.
- **Pass:** At least one tag returns ≥ 1 item, and the portal drill matches the export order of magnitude.
- **Fail (real):** Export errors or the analyst cannot drill despite List Viewer → re-check the role and `Export-ContentExplorerData` availability.
- **Expected, not fail:** All tags return 0 in a genuinely unclassified estate → record as **baseline coverage**; reference the labeling/SIT solutions.

Test 5 — Activity Explorer / audit returns activity

- **Procedure:** Perform a known labeling action on a pilot item (apply/change a sensitivity label), wait

for ingestion, then run `CLSREPORT-Report.ps1`; open `activity-summary.csv` and filter Activity Explorer to that user/label.

- **Pass:** The action appears as an activity row within the ingestion window.
- **Fail (real):** No activity ever appears though ingestion is on and time has passed → re-check audit

config and the record-type filters.

- **Expected, not fail:** No rows immediately after enabling ingestion → **ingestion lag**; re-test later.

Test 6 — Dashboard renders both views

- **Procedure:** Load `coverage-by-tag.csv` and `activity-summary.csv` into the chosen dashboard (Excel

workbook or Power BI).

- **Pass:** The **Coverage** and **Activity** views render with the expected columns and refresh cleanly.
- **Fail:** Column/shape mismatch → confirm the CSVs came from the current `CLSREPORT-Report.ps1` (the

dashboard binds to its column names).

Test 7 — Revocation works (rollback proof)

- **Procedure:** Remove a test analyst from a role group (`Remove-RoleGroupMember`), have them refresh

the portal.

- **Pass:** They lose the corresponding view (List and/or Content) after sign-in/token refresh.
- **Fail:** Access persists → confirm the removal targeted the right role group and the session was

refreshed.

Test 8 — Scheduled refresh + health check

- **Procedure:** Confirm the scheduled `CLSREPORT-Report.ps1` and `CLSREPORT-ScopeHealth.ps1` runs

produced fresh output and no drift/lag alerts.

- **Pass:** Fresh exports on cadence; health check clean (no unexpected Content Viewer members, audit

current, tag inventory stable).

- **Fail:** Stale exports or a drift/lag alert → investigate the schedule and the flagged condition.

Exit criteria (all must hold, with the two expected-states allowed)

- Both explorers available (or the licensing gate documented).
- Audit ingestion **True**.
- Role membership least-privilege and correct (Content Viewer minimal).
- Coverage export returns data **or** an explicit baseline note.
- Activity export returns data **or** an explicit ingestion-lag note.
- Dashboard renders both views; revocation proven; schedule + health check green.

6. Troubleshooting Guide — Classification Reporting & Analytics (Microsoft Purview)

Field issues and fixes. Most stem from **licensing/availability** (Content Explorer content viewing, Activity Explorer, and `Export-ContentExplorerData` are plan-gated and the portal is evolving), **audit ingestion being off or still catching up**, an **empty/baseline estate** (little is classified yet — a legitimate finding), or a **tenant-specific audit RecordType / Content Explorer TagType** that differs from the default. When a portal path or cmdlet differs from this guide, confirm the current equivalent rather than assuming. The five scripts are `CLSREPORT-Discovery.ps1`, `CLSREPORT-Configure.ps1`, `CLSREPORT-Validate.ps1`, `CLSREPORT-Report.ps1`, and `CLSREPORT-ScopeHealth.ps1`.

Content Explorer or Activity Explorer is missing, or shows only aggregate cards

- **Likely cause:** Licensing (Requirement 2.1) or role membership (Requirement 2.2). Item-level content viewing and Activity Explorer generally require **Microsoft 365 E5 / E5 Compliance**; aggregate cards are more broadly available.
- **Fix:** Confirm the subscription/add-ons in the M365 admin center and your role in Purview → Roles & scopes. Run `CLSREPORT-Discovery.ps1` to see which licensing-dependent signals your session can return. If the plan genuinely lacks the surface, note the gate in the report footnote and report on what is available — do not treat aggregate-only as a bug. **Verify your licensing/tenant.**

The explorer isn't where this guide says (menu path differs)

- **Cause:** Content Explorer and Activity Explorer are **moving between** the classic compliance portal (`compliance.microsoft.com → Data classification`) and the new Purview portal (`purview.microsoft.com → Information Protection / Data loss prevention`).
- **Fix:** Try both known locations (Implementation Step 7) and **verify against your current portal** — the surface has moved, the data is the same.

`CLSREPORT-Configure.ps1` made no change

- **Cause:** It is **safe-by-default** — without `-Commit` it runs in preview (`-WhatIf`) mode and only prints what it *would* do.

- **Fix:** Read the previewed actions, obtain change-control approval, then re-run with `-Commit`. Confirm

you also passed the intended switches (`-EnableAudit`, `-GrantListViewer`, `-GrantContentViewer`) and `-Members`.

Audit ingestion shows False (Activity Explorer / activity export empty)

- **Cause:** Unified audit log ingestion is off (Requirement 2.4) — Activity Explorer and

`Search-UnifiedAuditLog` have no source.

- **Fix:** `CLSREPORT-Configure.ps1 -EnableAudit -WhatIf` to preview, then `-Commit` under change control

(`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`). **Then wait** — allow the pipeline to backfill before judging activity coverage. `CLSREPORT-Validate.ps1` and `CLSREPORT-ScopeHealth.ps1` report the current state.

`Search-UnifiedAuditLog` returns nothing or errors

- **Cause:** Not connected to Exchange Online, ingestion off, the window predates ingestion being on, or an

invalid RecordType.

- **Fix:** `Connect-ExchangeOnline`; confirm ingestion True; widen `-Days` within the retained window; and

confirm the `-RecordTypes` values are current for your tenant (they evolve — **verify** in Purview → Audit). Note audit data has ingestion latency — very recent activity may not appear yet.

`CLSREPORT-Report.ps1` coverage-by-tag.csv is empty or all VERIFY

- **Cause 1 — export cmdlet absent:** `Export-ContentExplorerData` is licensing-/version-dependent and does

not resolve in every tenant. **Fix:** The script falls back to the **published tag inventory** and flags `VERIFY (portal)` — capture the Content Explorer counts from the **portal** for those tags, and note **"verify your licensing/tenant"** (Req 2.1).

- **Cause 2 — baseline estate:** Genuinely little is classified. **Fix:** Record as **baseline coverage** (a

finding to route to governance), reference the labeling/SIT solutions — not a failure (Req 2.5).

`CLSREPORT-Validate.ps1` reports FAIL on role membership

- **Cause:** The expected analysts aren't in **List Viewer**, or **Content Viewer** exceeds the minimal

threshold / contains an unexpected member (role-group drift).

- **Fix:** For missing List Viewer members, grant via `CLSREPORT-Configure.ps1 -GrantListViewer -Members …

-Commit. For an over-populated or drifted **Content Viewer** (which exposes sensitive data), remove the unexpected members with Remove-RoleGroupMember

-Identity "Content Explorer Content Viewer" -Member <upn>` and investigate how they were added.

An analyst can see item lists but not open content (or vice versa)

- **Cause:** Purview separates the two viewer roles — `Content Explorer List Viewer` shows the list/

location; `Content Explorer Content Viewer` opens the item content.

- **Fix:** Grant the specific role the task needs (least-privilege). Content Viewer exposes real sensitive

data — grant it only to the minimal approved set (Req 2.2), and only after change-control approval.

Revocation didn't take effect

- **Cause:** Role-group changes apply on the next sign-in / token refresh.
- **Fix:** Have the user sign out and back in (or wait for token refresh), then re-run

`CLSREPORT-Validate.ps1`. Confirm the removal targeted the correct role group.

`CLSREPORT-ScopeHealth.ps1` shows a workload "not reporting in"

- **Cause:** That workload produced no events for the RecordType in the window — could be no activity,

ingestion lag, or a RecordType that doesn't match how your tenant logs.

- **Fix:** Confirm classification/DLP activity has actually occurred on that workload; widen `-Days`; and

confirm the `-RecordTypes` are current (**verify** in Purview → Audit). An empty window shortly after enabling ingestion is **lag**, not a failure.

`Export-ContentExplorerData` / `Export-ActivityExplorerData` "not recognized"

- **Cause:** The export cmdlet is unlicensed or not present in this tenant/module version.
- **Fix:** Confirm E5 / E5 Compliance and the reporting role (Req 2.1 / 2.2). Where the cmdlet is genuinely

absent, use the **Content Explorer / Activity Explorer portal** for that data and flag **"verify your licensing/tenant"** — the reporting still stands, sourced from the portal plus `Search-UnifiedAuditLog`.

The dashboard tiles broke after a script run

- **Cause:** The dashboard binds to the CSV **column names** written by `CLSREPORT-Report.ps1`

(`coverage-by-tag.csv`, `activity-summary.csv`); a shape change breaks the binding.

- **Fix:** Confirm the CSVs came from the **current** `CLSREPORT-Report.ps1` and re-point the dashboard at

its column names. Keep the dashboard bound to the script output, not to a fixed portal export.

The solution "changed" something

- **It should not have.** Only `CLSREPORT-Configure.ps1` writes, only with `-Commit`, and only to (a) audit

ingestion config and (b) viewer role-group membership — never to content, labels, SITs, or policy. If you believe more changed, run `CLSREPORT-Validate.ps1`, review the role-group membership, and remove any unintended grant with `Remove-RoleGroupMember`.

Escalation boundary

Regulatory, records-retention, and legal questions — *which* content must be relabeled or retained, whether an activity is a *policy violation*, legal-hold implications — are **not** resolved here. Capture the technical evidence (the coverage and activity exports, the validation output) and route the determination to your compliance, records, privacy, and legal functions (GR §5). Standing up label/DLP **enforcement** is a separate, change-controlled solution, not part of this reporting implementation.

7. Best Practices — Classification Reporting & Analytics (Microsoft Purview)

- **Report; don't rule.** This solution makes the *existing* classification signal visible. A low coverage

number, a suspicious egress pattern, or an under-labeled site is a **finding to route to governance** — *which* content must be relabeled, retained, or investigated is a compliance/records/privacy/legal call (GR §5). Hand over the evidence; let the owning function own the determination.

- **Separate "seeing that content exists" from "reading it."** Grant `Content Explorer List Viewer`

broadly to analysts who need coverage numbers, and `Content Explorer Content Viewer` — which exposes real sensitive data — only to the minimal approved set. Keep both separate from the people who *author* labels/DLP and from records/legal. `CLSREPORT-Configure.ps1` grants each role independently for exactly this reason.

- **Preview every governed change before you commit it.** `CLSREPORT-Configure.ps1` defaults to preview;

read every intended audit-enable and role-grant action, then re-run with `-Commit` under change control. Enabling audit ingestion and granting Content Viewer are the only two changes this solution makes — treat them accordingly.

- **Pilot before the full audience.** Grant the viewer roles to a small named analyst subset, prove both

explorers return data and the dashboards render, then expand to the full `Purview Classification Reporting - Analysts` group. Reporting is low-risk, but the sensitive-data grant is not — earn the broad grant with a validated pilot.

- **Keep auditing on, and give it time.** Activity Explorer and the activity exports are only as complete

as the unified audit log feeding them. Enable ingestion early, and after enabling it, allow the pipeline to backfill before judging activity coverage — an empty window right after enablement is **ingestion lag**, not a failure.

- **Treat an empty/baseline estate as a valid finding.** A tenant with few published labels or SIT

detections will show near-empty explorers. Record that as **baseline coverage** and reference the labeling/SIT solutions — it is the honest starting number, not a broken report.

- **Verify availability; never assume it.** Content-item viewing, Activity Explorer, trainable classifiers,

and `Export-ContentExplorerData` are licensing-gated and the portal is moving surfaces between the classic compliance portal and the new Purview portal. When a blade or cmdlet isn't where this guide says, confirm the current equivalent in your tenant and flag **"verify your licensing/tenant"** in the report footnote rather than forcing a path that isn't there.

- **Parameterize the volatile values.** Audit **RecordTypes** and Content Explorer **TagTypes** evolve and

differ across tenants. Pass `-RecordTypes` / `-TagTypes` after confirming the current values in Purview → Audit rather than trusting a default blindly — the scripts parameterize these precisely because they drift.

- **Bind the dashboard to the export shape, not the portal screenshot.** The Coverage and Activity views

bind to the column names `CLSREPORT-Report.ps1` writes (`coverage-by-tag.csv`, `activity-summary.csv`). Keep the dashboard pointed at the current script output so a column rename never silently breaks a tile.

- **Make it recurring, and watch for drift.** Schedule `CLSREPORT-Report.ps1` and `CLSREPORT-ScopeHealth.ps1`

(e.g. weekly) so the numbers refresh and you catch **role-group drift** (an unexpected new Content Viewer), **audit lag**, a **workload dropping out** of reporting, and **tag-inventory changes** that reshape the report — classification coverage is a trend to watch, not a one-time snapshot.

- **Keep it reproducible.** Retain the dated CSV exports as the baseline; each later run compares against it

to show whether coverage is trending up and activity is within expectation as labeling (separately) expands.

8. Deliverables Summary — Classification Reporting & Analytics (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What the reporting delivers (coverage via Content Explorer, activity/trend via Activity Explorer), how the signal is produced, the availability/licensing caveat, the reporting-vs-enforcement boundary, the outcome-vs-determination boundary, and the naming convention

Document	Purpose
2. Requirements Guide	Every reporting prerequisite with Verify / Implement-if-missing (compliance licensing, the two Content Explorer viewer role groups, a named analyst group + role separation, audit ingestion, existing classification signal, pilot scope + change control, PowerShell toolchain)
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal + PowerShell steps: baseline readiness, confirm the signal, design the analyst group + dashboard, enable audit + the scoped viewer roles (pilot first), open the explorers, produce pilot reports, validate, expand to the full audience, schedule exports + health checks
5. Testing Guide	Validation procedures with explicit pass/fail, treating empty/baseline estate and ingestion lag as expected states
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for reporting on classification safely and least-privilege

PowerShell scripts

All are safe-by-default (read-only, or `-WhatIf` preview unless `-Commit`), use `Connect-IPPSSession` and/or `Connect-ExchangeOnline`, and store **no** credentials.

Script	What it does
`CLSREPORT-Discovery.ps1`	Read-only readiness baseline: whether unified audit log ingestion is on, current membership of the two Content Explorer viewer role groups, an inventory count of sensitivity/retention labels and SITs available to report on, and whether the export cmdlets resolve in the tenant. Run it first (Implementation Phase 1)
`CLSREPORT-Configure.ps1`	Configuration helper for the two governed prerequisites — enables unified audit log ingestion (`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`) and grants the scoped viewer roles (`Add-RoleGroupMember` for `Content Explorer List Viewer` / `Content Explorer Content Viewer`). Defaults to preview; writes only with `-Commit`; Content Viewer flagged as sensitive-data grant. Creates no labels/SITs/policies

Script	What it does
`CLSREPORT-Validate.ps1`	Read-only PASS/FAIL gate: audit ingestion True, expected analysts present in List Viewer, Content Viewer within the minimal threshold with no drift, and the export surfaces resolve (unresolved `Export-ContentExplorerData` reported as VERIFY, not FAIL). Run before expanding to the full audience
`CLSREPORT-Report.ps1`	Read-only export of the two dashboard feeds: COVERAGE (items per label/retention label/SIT/classifier via `Export-ContentExplorerData` where available — flagged "verify in your tenant", portal fallback otherwise) to `coverage-by-tag.csv`, and ACTIVITY (label lifecycle + DLP + egress via `Search-UnifiedAuditLog`) to `activity-summary.csv`. Empty-estate and ingestion-lag noted as valid findings
`CLSREPORT-ScopeHealth.ps1`	Read-only reporting data-source health: audit ingestion + retention window, per-workload activity presence (is each workload reporting in), the tag inventory to report on, and licensing-readiness flags for the gated surfaces (Content Viewer, Activity Explorer export). Catches audit lag, workload drop-off, and tag drift on a schedule

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess → Design → Configure → Pilot (simulate/limited scope)

→ Validate → Expand**, with explicit pass/fail gates and a documented rollback.

Outcome

Classification made **visible, measurable, and defensible** in Microsoft Purview: coverage reported across every sensitivity label, retention label, SIT, and classifier; activity reported across label lifecycle, DLP matches, and egress; dashboards standing on repeatable CSV exports; and the access model least-privilege and audited — every step read-only or previewed before commit, changing no content. Standing up label/DLP *enforcement* is a separate, scoped solution. Regulatory, records, and legal determinations arising from the reports remain with the organization's governance functions; this solution delivers the technical method and the evidence.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CLSREPORT-Configure.ps1

Purpose: Turns on the two governed prerequisites the reporting depends on: (1) unified audit log ingestion (Activity Explorer's data source) via Set-AdminAuditLogConfig, and (2) least-privilege Content Explorer viewer role-group membership (Content Explorer List Viewer for coverage reporting; Content Explorer Content Viewer only for those who must open sensitive items) via Add-RoleGroupMember. Creates NO labels, SITs, or policies — it grants sight of the existing classification signal. HOW TO RUN Preview every intended change (default, no writes): powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Configure.ps1 -EnableAudit -GrantListViewer -Members user1@contoso.com,user2@contoso.com -WhatIf Apply after change-control approval: powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Configure.ps1 -EnableAudit -GrantListViewer -Members user1@contoso.com -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline) via the ExchangeOnlineManagement module. PowerShell 5.1+ or 7+. EXPECTED RESULTS With -WhatIf (default): prints each action it WOULD take, makes no change. With -Commit: enables audit ingestion (if -EnableAudit) and adds the named members to the requested viewer role group(s). PASS signal: re-running CLSREPORT-Validate.ps1 shows audit ingestion True and the intended members present, with Content Viewer minimal. .NOTES Purview — Classification Reporting & Analytics. Writes gated by -Commit (default is -WhatIf preview). No credentials in code. Expected results: audit ingestion True + intended viewer-role membership; Content Viewer kept minimal (it exposes sensitive content). > [CmdletBinding()] param([switch]$EnableAudit, [switch]$GrantListViewer, [switch]$GrantContentViewer, [string[]]$Members = @(), [string]$ListViewerRoleGroup = 'Content Explorer List Viewer', [string]$ContentViewerRoleGroup = 'Content Explorer Content Viewer', [switch]$Commit, [switch]$WhatIf, [string]$UserPrincipalName)

How to run: Preview every intended change (default, no writes): powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Configure.ps1 -EnableAudit -GrantListViewer -Members user1@contoso.com,user2@contoso.com -WhatIf Apply after change-control approval: powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Configure.ps1 -EnableAudit -GrantListViewer -Members user1@contoso.com -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline) via the ExchangeOnlineManagement module. PowerShell 5.1+ or 7+. EXPECTED RESULTS With -WhatIf (default): prints each action it WOULD take, makes no change. With -Commit: enables audit ingestion (if -EnableAudit) and adds the named members to the requested viewer role group(s). PASS signal: re-running CLSREPORT-Validate.ps1 shows audit ingestion True and the intended members present, with Content Viewer minimal. .NOTES Purview — Classification Reporting & Analytics. Writes gated by -Commit (default is -WhatIf preview). No credentials in code. Expected results: audit ingestion True + intended viewer-role membership; Content Viewer kept minimal (it exposes sensitive content). > [CmdletBinding()] param([switch]$EnableAudit, [switch]$GrantListViewer, [switch]$GrantContentViewer, [string[]]$Members = @(), [string]$ListViewerRoleGroup = 'Content Explorer List Viewer', [string]$ContentViewerRoleGroup = 'Content Explorer Content Viewer', [switch]$Commit, [switch]$WhatIf, [string]$UserPrincipalName)

CLSREPORT-Discovery.ps1

Purpose: Builds the reporting baseline before you enable anything: whether unified audit log ingestion (Activity Explorer's data source) is on; current membership of the two Content Explorer viewer role groups; an inventory count of sensitivity labels, retention labels, and sensitive information types available to report on; and whether the export cmdlets this solution uses resolve in your tenant. Read-only — makes no changes. Run it first (Implementation Phase 1). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Discovery.ps1 -CsvPath .\CLSREPORT-out\readiness.csv Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline) via the ExchangeOnlineManagement module. PowerShell 5.1+ or 7+. EXPECTED RESULTS Console tables for audit-ingestion state, the two viewer role groups' members, and the classification tag inventory, plus an optional CSV. PASS signal: audit ingestion is True, the role groups exist with a known (minimal) membership, and a non-empty tag inventory is available to report on. A zero tag inventory is a valid BASELINE finding (largely unclassified estate), not a failure. .NOTES Purview — Classification Reporting & Analytics. Read-only against S&C PowerShell / Exchange Online. No changes, no credentials in code. Expected results: PASS = audit True + viewer roles known + tag inventory present (or explicit baseline note). > [CmdletBinding()] param([string]$CsvPath, [string]$ListViewerRoleGroup = 'Content Explorer List Viewer', [string]$ContentViewerRoleGroup = 'Content Explorer Content Viewer', [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Discovery.ps1 -CsvPath .\CLSREPORT-out\readiness.csv Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline) via the ExchangeOnlineManagement module. PowerShell 5.1+ or 7+. EXPECTED RESULTS Console tables for audit-ingestion state, the two viewer role groups' members, and the classification tag inventory, plus an optional CSV. PASS signal: audit ingestion is True, the role groups exist with a known (minimal) membership, and a non-empty tag inventory is available to report on. A zero tag inventory is a valid BASELINE finding (largely unclassified estate), not a failure. .NOTES Purview — Classification Reporting & Analytics. Read-only against S&C PowerShell / Exchange Online. No changes, no credentials in code. Expected results: PASS = audit True + viewer roles known + tag inventory present (or explicit baseline note). > [CmdletBinding()] param([string]$CsvPath, [string]$ListViewerRoleGroup = 'Content Explorer List Viewer', [string]$ContentViewerRoleGroup = 'Content Explorer Content Viewer', [string]$UserPrincipalName)

CLSREPORT-Report.ps1

Purpose: Produces the two repeatable dashboard feeds: (1) COVERAGE — items per sensitivity label / retention label / SIT / trainable classifier, via Export-ContentExplorerData WHERE that cmdlet is available in your tenant (flagged VERIFY otherwise, with the Content Explorer portal as the fallback); and (2) ACTIVITY — sensitivity-label lifecycle, DLP matches, and egress activity over a rolling window, via Search-UnifiedAuditLog. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Report.ps1 -Days 30 -OutputFolder .\CLSREPORT-out Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline). PowerShell 5.1+ or 7+. EXPECTED RESULTS Writes coverage-by-tag.csv (or an explicit VERIFY note if Export-ContentExplorerData is absent) and activity-summary.csv into -OutputFolder. PASS signal: at least one tag returns >= 1 item (or an explicit BASELINE note for an unclassified estate) and >= 1 classification/DLP activity row (or an explicit ingestion-lag note). Load the CSVs into Excel or Power BI to render the Coverage/Activity views. .NOTES Purview — Classification Reporting & Analytics. Read-only against S&C PowerShell / Exchange Online. No changes, no credentials in code. Expected results: coverage + activity CSVs written; empty-estate and ingestion-lag are valid findings, not failures. Export-ContentExplorerData availability is licensing-/tenant-dependent — VERIFY in

your tenant. > [CmdletBinding()] param([int]$Days = 30, [string]$OutputFolder = '.\CLSREPORT-out', [ValidateSet('SensitiveInformationType','SensitivityLabel','RetentionLabel','TrainableClassifier')] [string[]]$TagTypes = @('SensitivityLabel','RetentionLabel','SensitiveInformationType'), Audit record types that surface classification/DLP/egress activity (parameterized — values evolve; VERIFY in your tenant). [string[]]$RecordTypes = @('SensitivityLabelAction','SensitivityLabeledFileAction','ComplianceDLPSharePoint','ComplianceDLPExchange','DLPEndpoint'), [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Report.ps1 -Days 30 -OutputFolder .\CLSREPORT-out Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline). PowerShell 5.1+ or 7+. EXPECTED RESULTS Writes coverage-by-tag.csv (or an explicit VERIFY note if Export-ContentExplorerData is absent) and activity-summary.csv into -OutputFolder. PASS signal: at least one tag returns >= 1 item (or an explicit BASELINE note for an unclassified estate) and >= 1 classification/DLP activity row (or an explicit ingestion-lag note). Load the CSVs into Excel or Power BI to render the Coverage/Activity views. .NOTES Purview — Classification Reporting & Analytics. Read-only against S&C PowerShell / Exchange Online. No changes, no credentials in code. Expected results: coverage + activity CSVs written; empty-estate and ingestion-lag are valid findings, not failures. Export-ContentExplorerData availability is licensing-/tenant-dependent — VERIFY in your tenant. > [CmdletBinding()] param([int]$Days = 30, [string]$OutputFolder = '.\CLSREPORT-out', [ValidateSet('SensitiveInformationType','SensitivityLabel','RetentionLabel','TrainableClassifier')] [string[]]$TagTypes = @('SensitivityLabel','RetentionLabel','SensitiveInformationType'), Audit record types that surface classification/DLP/egress activity (parameterized — values evolve; VERIFY in your tenant). [string[]]$RecordTypes = @('SensitivityLabelAction','SensitivityLabeledFileAction','ComplianceDLPSharePoint','ComplianceDLPExchange','DLPEndpoint'), [string]$UserPrincipalName)

CLSREPORT-ScopeHealth.ps1

Purpose: Ongoing health of what the reporting depends on, so a dashboard's numbers stay trustworthy: whether unified audit log ingestion is on and the audit retention window; which workloads are actually producing classification/DLP activity (so an empty view is explained by "no signal" vs "not reporting in"); the published tag inventory the explorers break down by; and licensing-readiness flags for the licensing-gated surfaces (Content Viewer, Activity Explorer export). Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CLSREPORT-ScopeHealth.ps1 -Days 30 -CsvPath .\CLSREPORT-out\scope-health.csv Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline). PowerShell 5.1+ or 7+. EXPECTED RESULTS A health table: audit ingestion state, audit retention (where readable), per-workload activity presence, tag-inventory counts, and licensing-readiness flags. PASS signal: ingestion True, a non-empty tag inventory, and at least one workload reporting activity in the window. Empty inventory (baseline estate) and unavailable licensing-gated surfaces are VERIFY/BASELINE notes, not failures. .NOTES Purview — Classification Reporting & Analytics. Read-only against S&C PowerShell / Exchange Online. No changes, no credentials in code. Expected results: audit True + tag inventory present + a workload reporting in; licensing gates and baseline estate flagged VERIFY/BASELINE, not FAIL. > [CmdletBinding()] param([int]$Days = 30, [string]$CsvPath, [string[]]$RecordTypes = @('SensitivityLabelAction','SensitivityLabeledFileAction','ComplianceDLPSharePoint','ComplianceDLPExchange','DLPEndpoint'), [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\CLSREPORT-ScopeHealth.ps1 -Days 30 -CsvPath .\CLSREPORT-out\scope-health.csv Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline). PowerShell 5.1+ or

7+. EXPECTED RESULTS A health table: audit ingestion state, audit retention (where readable), per-workload activity presence, tag-inventory counts, and licensing-readiness flags. PASS signal: ingestion True, a non-empty tag inventory, and at least one workload reporting activity in the window. Empty inventory (baseline estate) and unavailable licensing-gated surfaces are VERIFY/BASELINE notes, not failures. .NOTES Purview — Classification Reporting & Analytics. Read-only against S&C PowerShell / Exchange Online. No changes, no credentials in code. Expected results: audit True + tag inventory present + a workload reporting in; licensing gates and baseline estate flagged VERIFY/BASELINE, not FAIL. > [CmdletBinding()] param([int]$Days = 30, [string]$CsvPath, [string[]]$RecordTypes = @('SensitivityLabelAction','SensitivityLabeledFileAction','ComplianceDLPSharePoint','ComplianceDLPExchange','DLPEndpoint'), [string]$UserPrincipalName)

CLSREPORT-Validate.ps1

Purpose: Confirms the reporting prerequisites are in place before you expand to the full analyst audience: unified audit log ingestion is True; the intended analysts are in Content Explorer List Viewer; Content Explorer Content Viewer is minimal (it exposes sensitive content); and the export surfaces this solution relies on resolve in the tenant. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Validate.ps1 -ExpectedListViewer user1@contoso.com,user2@contoso.com -MaxContentViewers 3 Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline). PowerShell 5.1+ or 7+. EXPECTED RESULTS A PASS/FAIL table across the checks. PASS = audit ingestion True, expected analysts present in List Viewer, Content Viewer within the minimal threshold, and no unexpected members. An unresolved Export-ContentExplorerData is reported as VERIFY (licensing/tenant-dependent), not FAIL. .NOTES Purview — Classification Reporting & Analytics. Read-only against S&C PowerShell / Exchange Online. No changes, no credentials in code. Expected results: all applicable checks PASS; Content Viewer minimal; export availability flagged VERIFY where tenant-dependent. > [CmdletBinding()] param([string[]]$ExpectedListViewer = @(), [string[]]$AllowedContentViewer = @(), [int]$MaxContentViewers = 5, [string]$ListViewerRoleGroup = 'Content Explorer List Viewer', [string]$ContentViewerRoleGroup = 'Content Explorer Content Viewer', [string]$CsvPath, [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\CLSREPORT-Validate.ps1 -ExpectedListViewer user1@contoso.com,user2@contoso.com -MaxContentViewers 3 Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online PowerShell (Connect-ExchangeOnline). PowerShell 5.1+ or 7+. EXPECTED RESULTS A PASS/FAIL table across the checks. PASS = audit ingestion True, expected analysts present in List Viewer, Content Viewer within the minimal threshold, and no unexpected members. An unresolved Export-ContentExplorerData is reported as VERIFY (licensing/tenant-dependent), not FAIL. .NOTES Purview — Classification Reporting & Analytics. Read-only against S&C PowerShell / Exchange Online. No changes, no credentials in code. Expected results: all applicable checks PASS; Content Viewer minimal; export availability flagged VERIFY where tenant-dependent. > [CmdletBinding()] param([string[]]$ExpectedListViewer = @(), [string[]]$AllowedContentViewer = @(), [int]$MaxContentViewers = 5, [string]$ListViewerRoleGroup = 'Content Explorer List Viewer', [string]$ContentViewerRoleGroup = 'Content Explorer Content Viewer', [string]$CsvPath, [string]$UserPrincipalName)